

GraphQL versus REST: Um Experimento Controlado sobre Tempo de Resposta e Tamanho de Payload

Luiz Paulo Saud
Engenharia de Software
PUC Minas
Belo Horizonte, Brasil

Arthur Curi
Engenharia de Software
PUC Minas
Belo Horizonte, Brasil

Helio Teixeira
Engenharia de Software
PUC Minas
Belo Horizonte, Brasil

Resumo—GraphQL surge como alternativa às APIs REST, prometendo consultas mais flexíveis e eficientes ao permitir que o cliente especifique exatamente os dados desejados. Entretanto, os benefícios reais dessa adoção ainda carecem de evidência empírica controlada. Este trabalho apresenta um experimento controlado que compara as duas abordagens quanto a duas variáveis dependentes: tempo de resposta e tamanho do payload. Ambos os paradigmas foram expostos sobre a mesma base de dados e a mesma infraestrutura, isolando o estilo de API como única variável independente. Foram executadas 200 medições para cada combinação de quatro cenários de consulta e dois tratamentos, totalizando 1.600 observações. Os resultados, validados por testes não paramétricos de Mann–Whitney com tamanho de efeito de Cliff, mostram um cenário matizado: GraphQL reduz o tempo de resposta em até 68% e o tamanho do payload em até 84% em consultas aninhadas e com seleção de campos, mas impõe sobrecarga de processamento em consultas simples, nas quais REST se mostrou mais rápido. Conclui-se que a superioridade de GraphQL é dependente do padrão de acesso aos dados, e não absoluta.

Index Terms—GraphQL, REST, APIs Web, experimento controlado, desempenho, engenharia de software experimental.

I. INTRODUÇÃO

A linguagem de consulta GraphQL, proposta pelo Facebook como metodologia de implementação de APIs Web, representa uma alternativa às populares APIs REST. Baseada em grafos, a linguagem permite que clientes consultem a base de dados a partir de *schemas* tipados, especificando, num formato definido pelo fornecedor da API, exatamente quais campos desejam receber. Em contraste, APIs REST baseiam-se em *endpoints*: operações pré-definidas, orientadas a recursos, que retornam representações fixas dos objetos.

Desde o seu surgimento, diversos sistemas migraram de REST para GraphQL, mantendo compatibilidade, mas buscando os benefícios da nova linguagem. Entretanto, **não está claro quais os reais benefícios da adoção de uma API GraphQL em detrimento de uma API REST**. Duas limitações recorrentes das APIs REST motivam essa discussão: o *over-fetching* (o servidor devolve mais campos do que o cliente precisa) e o problema *N+1* (a obtenção de recursos relacionados exige múltiplas requisições). GraphQL promete mitigar ambos.

O objetivo deste trabalho é realizar um experimento controlado para avaliar quantitativamente esses benefícios, respondendo a duas questões de pesquisa:

- **RQ1.** Respostas às consultas GraphQL são mais rápidas que respostas às consultas REST?
- **RQ2.** Respostas às consultas GraphQL têm tamanho menor que respostas às consultas REST?

II. DESENHO DO EXPERIMENTO

Esta seção formaliza o desenho experimental seguindo os princípios da engenharia de software experimental.

A. Hipóteses

Para cada questão de pesquisa formulam-se hipóteses nula (H_0) e alternativa (H_1), avaliadas por cenário de consulta:

• RQ1 – Tempo de resposta:

- H_{10} : não há diferença no tempo de resposta entre GraphQL e REST ($\mu_{tempo}^{GraphQL} = \mu_{tempo}^{REST}$).
- H_{11} : há diferença no tempo de resposta entre GraphQL e REST ($\mu_{tempo}^{GraphQL} \neq \mu_{tempo}^{REST}$).

• RQ2 – Tamanho da resposta:

- H_{20} : não há diferença no tamanho do payload entre GraphQL e REST ($\mu_{tam}^{GraphQL} = \mu_{tam}^{REST}$).
- H_{21} : há diferença no tamanho do payload entre GraphQL e REST ($\mu_{tam}^{GraphQL} \neq \mu_{tam}^{REST}$).

Adota-se nível de significância $\alpha = 0,05$.

B. Variáveis Dependentes

- **Tempo de resposta** (*latency_ms*): tempo de ponta a ponta, em milissegundos, desde o envio da(s) requisição(ões) até o recebimento completo da(s) resposta(s). Para REST, em cenários aninhados, contabiliza-se o tempo somado de todas as requisições necessárias.
- **Tamanho da resposta** (*size_bytes*): número total de bytes do(s) corpo(s) de resposta recebido(s).

C. Variáveis Independentes

A variável independente principal é o **estilo de API**, com dois níveis: *REST* e *GraphQL*. Um segundo fator de bloqueio é o **cenário de consulta**, com quatro níveis (descritos na Seção II-E), introduzido para verificar se o efeito do estilo de API depende do padrão de acesso aos dados.

D. Tratamentos

Os tratamentos resultam do cruzamento dos níveis: para cada cenário, executa-se a mesma operação lógica via *REST* e via *GraphQL*. Como ambos compartilham base de dados, processo e *stack*, a única diferença sistemática entre os tratamentos é o paradigma de API.

E. Objetos Experimentais

O objeto experimental é uma base relacional sintética e determinística (semente fixa) com 50 usuários, 250 *posts* (5 por usuário) e 2.000 comentários (8 por *post*), modelando o relacionamento *usuário* $\rightarrow$ *post* $\rightarrow$ *comentário*. Sobre ela definem-se quatro cenários:

- **C1 – Recurso único completo:** obter um usuário com todos os campos. Cenário de controle, sem *over-fetching* nem N+1.
- **C2 – Recurso único parcial:** obter apenas 3 campos de um usuário. Expõe o *over-fetching* do *REST*.
- **C3 – Consulta aninhada (N+1):** obter um usuário, seus *posts* e os comentários de cada *post*. Expõe o problema N+1 do *REST* ($1 + 1 + 5 = 7$ requisições) contra uma única requisição *GraphQL*.
- **C4 – Coleção:** obter os 50 usuários completos. Cenário de controle para grandes volumes sem aninhamento.

F. Tipo de Projeto Experimental

Trata-se de um projeto *fatorial* 2×4 (estilo de API $\times$ cenário) com medições repetidas, executado em ambiente intra-sujeito (mesma máquina e mesma base para todos os tratamentos).

G. Quantidade de Medições

Cada combinação (cenário $\times$ estilo) foi medida 200 vezes, precedidas de 20 execuções de aquecimento (descartadas) para estabilizar *cache*, conexões persistentes e *JIT*. O total é $4 \times 2 \times 200 = 1.600$ observações, com requisições intercaladas entre os estilos para diluir efeitos temporais.

H. Ameaças à Validade

Validade interna: ruído do sistema operacional e variação de agendamento podem afetar as medições; mitigado por aquecimento, medições repetidas, intercalação dos tratamentos e uso de estatística robusta. **Validade de construção:** o tamanho do payload é medido em bytes do corpo da resposta, ignorando *headers* e compressão; o tempo inclui o processamento do servidor, refletindo a experiência real do cliente. **Validade externa:** o uso de uma base sintética e de comunicação local (*localhost*) elimina a latência de rede, podendo subestimar o ganho do *GraphQL* em redes reais (onde reduzir o número

de viagens e o volume de bytes é ainda mais relevante).

Validade de conclusão: verificou-se a normalidade (Shapiro–Wilk) e, dada a violação frequente, optou-se por testes não paramétricos.

III. PREPARAÇÃO E EXECUÇÃO

A infraestrutura foi implementada em Python. Um único servidor (*Flask*) expõe a *mesma* base de dados por duas vias: rotas *REST* orientadas a recursos e um *endpoint GraphQL* (*Ariadne*) com *schema* tipado e resolvedores aninhados. Um cliente (*requests*, com sessão *keep-alive*) aplica os tratamentos e registra as variáveis dependentes em arquivo CSV. A Listagem 1 ilustra a consulta *GraphQL* do cenário C3.

Listing 1. Consulta GraphQL do cenário aninhado C3.

```
query($id: ID!) {  
  user(id: $id) {  
    id name email  
    posts {  
      id title likes  
      comments { id name body }  
    }  
  }  
}
```

A. Ambiente de Execução

Os *trials* foram executados em: CPU Intel64 Family 6 (28 *threads* lógicas), 64 GB de RAM, Windows 10 (build 10.0.26200), Python 3.12.2, *Flask* 3.0, *Ariadne* 0.23 e comunicação via *loopback* (127.0.0.1). A semente aleatória (42) garante a reprodutibilidade da base de dados.

IV. RESULTADOS

A Tabela I consolida as medianas e o resultado dos testes de hipótese. Todos os testes de Mann–Whitney rejeitaram H_0 ($p \ll 0,001$) e apresentaram tamanho de efeito de Cliff classificado como *grande*, indicando diferenças estatística e praticamente relevantes em todos os cenários.

A. RQ1 – Tempo de Resposta

A Figura 1 apresenta a latência média (escala logarítmica) com intervalos de confiança de 95%, e a Figura 2 mostra a distribuição via *boxplots*. O resultado é **dependente do cenário**:

- Em consultas simples (C1, C2 e C4), *REST* foi significativamente mais rápido. A análise e validação do *schema GraphQL* adicionam uma sobrecarga fixa de processamento que, na ausência de latência de rede, domina o tempo total.
- No cenário aninhado C3, *GraphQL* foi **67,97% mais rápido** que *REST*. Aqui o problema N+1 penaliza fortemente o *REST*, que precisa de sete requisições sequenciais, enquanto o *GraphQL* resolve tudo em uma única viagem.

Portanto, rejeita-se H_{10} em todos os cenários, mas a *direção* do efeito favorece o *GraphQL* apenas quando há agregação de recursos relacionados.

Tabela I
MEDIANAS POR CENÁRIO E TRATAMENTO, REDUÇÃO PROPORCIONADA PELO GraphQL E SIGNIFICÂNCIA (MANN-WHITNEY). VALORES POSITIVOS DE REDUÇÃO INDICAM VANTAGEM DO GraphQL.

Cenário	Métrica	REST	GraphQL	Redução (%)	p-valor	Efeito
C1 – Recurso completo	Tempo (ms)	2,39	4,31	−80,5	< 0,001	grande
C2 – Recurso parcial	Tempo (ms)	2,26	3,76	−66,3	< 0,001	grande
C3 – Aninhada (N+1)	Tempo (ms)	16,42	5,26	+67,97	< 0,001	grande
C4 – Coleção	Tempo (ms)	2,64	5,96	−126,0	< 0,001	grande
C1 – Recurso completo	Tamanho (B)	554	574	−3,6	< 0,001	grande
C2 – Recurso parcial	Tamanho (B)	554	87	+84,3	< 0,001	grande
C3 – Aninhada (N+1)	Tamanho (B)	18.380	13.238	+27,98	< 0,001	grande
C4 – Coleção	Tamanho (B)	29.943	30.062	−0,4	< 0,001	grande

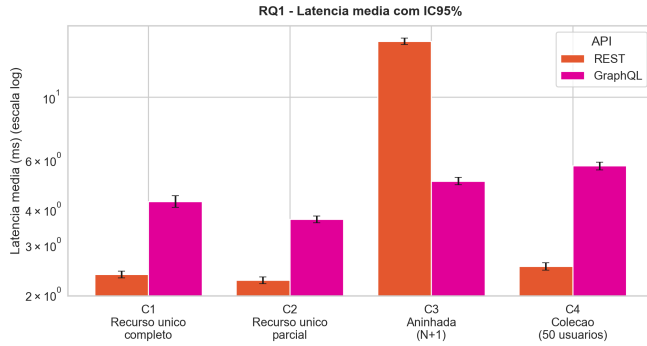


Figura 1. Latência média por cenário (escala log) com IC95%.

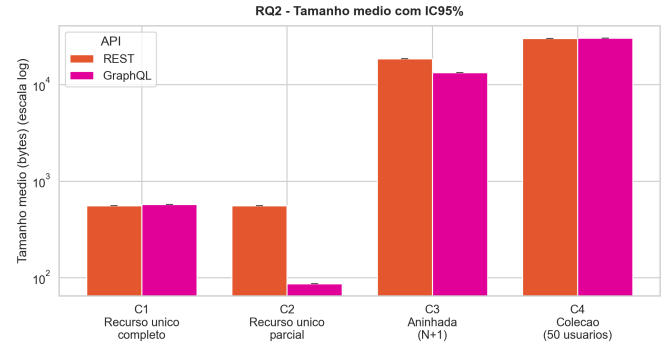


Figura 3. Tamanho médio do payload por cenário (escala log) com IC95%.

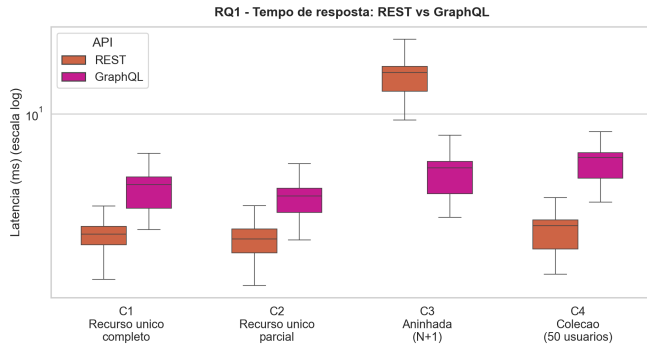


Figura 2. Distribuição da latência por cenário e tratamento.

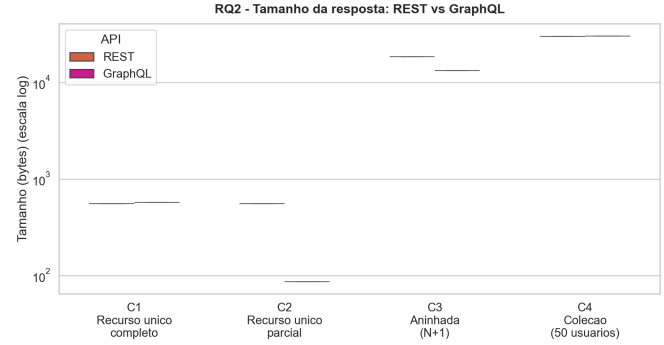


Figura 4. Distribuição do tamanho do payload por cenário e tratamento.

B. RQ2 – Tamanho da Resposta

As Figuras 3 e 4 resumem o tamanho dos *payloads*. Novamente o efeito depende do padrão de acesso:

- Em C2, a capacidade do GraphQL de selecionar campos reduziu o payload em **84,3%** (de 554 para 87 bytes), eliminando o *over-fetching* do REST.
- Em C3, a ausência de campos redundantes e de envelopes repetidos resultou em **27,98%** menos bytes para o GraphQL.
- Em C1 e C4, nos quais o cliente solicita todos os campos, os tamanhos são praticamente equivalentes; o GraphQL chega a ser ligeiramente maior devido ao envelope *data* da resposta.

Rejeita-se H_2_0 em todos os cenários, com vantagem expressiva do GraphQL sempre que o cliente não necessita do objeto completo.

V. DISCUSSÃO

Os dados desautorizam tanto a visão de que “GraphQL é sempre superior” quanto a de que “GraphQL é apenas *hype*”. A vantagem do GraphQL é **condicional ao padrão de acesso aos dados**:

GraphQL vence claramente quando (i) o cliente precisa de um subconjunto dos campos (combatendo *over-fetching*, RQ2/C2) ou (ii) precisa agregar recursos relacionados em uma única operação (combatendo o N+1, RQ1/C3). Esses são exatamente os casos que motivaram a criação da linguagem, e

os ganhos medidos (até -84% em bytes e -68% em tempo) são substanciais.

REST vence em consultas simples de recurso único ou coleção completa, nas quais não há over-fetching nem aninhamento. Nesses casos, a sobrecarga de *parsing* e validação do *schema* GraphQL não é compensada por nenhuma economia de viagens ou de dados.

Vale destacar que o experimento foi conduzido em *loop-back*, sem latência de rede. Em ambientes reais, o custo de cada viagem adicional (RTT) e de cada byte trafegado é muito maior; logo, espera-se que a vantagem do GraphQL nos cenários C2 e C3 seja *ainda mais pronunciada*, enquanto a desvantagem nos cenários simples tende a se diluir frente ao RTT de rede.

VI. CONCLUSÃO

Este experimento controlado comparou GraphQL e REST quanto a tempo de resposta e tamanho de payload, sob infraestrutura e base de dados idênticas, em 1.600 medições. Quanto à RQ1, GraphQL *não* é universalmente mais rápido: vence em consultas aninhadas (até 68% menos tempo), mas perde em consultas simples devido à sobrecarga de processamento. Quanto à RQ2, GraphQL produz respostas menores sempre que o cliente seleciona um subconjunto dos dados (até 84% menos bytes), sendo equivalente quando todos os campos são solicitados. A conclusão central é que a decisão entre GraphQL e REST deve ser guiada pelo **padrão de consumo de dados da aplicação**: APIs com clientes heterogêneos, telas ricas e dados altamente relacionados tendem a se beneficiar do GraphQL, enquanto serviços simples e orientados a recursos podem permanecer mais eficientes com REST. Como trabalhos futuros, sugere-se replicar o experimento sobre rede real, incluir compressão (*gzip*) e avaliar o custo de servidor (CPU/memória) sob carga concorrente.

REFERÊNCIAS

- [1] GraphQL Foundation, “GraphQL Specification,” 2021. [Online]. Disponível em: <https://spec.graphql.org/>
- [2] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Tese de Doutorado, Univ. of California, Irvine, 2000.
- [3] G. Brito and M. T. Valente, “REST vs GraphQL: A Controlled Experiment,” in *IEEE Int. Conf. on Software Architecture (ICSA)*, 2020.
- [4] C. Wohlin et al., *Experimentation in Software Engineering*. Springer, 2012.